

FUNCIONES NECESARIAS PARA LA WEB PUBLICA:

```
// supabase/functions/debacu-eval-login/index.ts
// deno-lint-ignore-file no-explicit-any
import "jsr:@supabase/functions-js/edge-runtime.d.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.45.4";
import { json, preflight } from "../_shared/cors.ts";
import { requireUser } from "../_shared/auth.ts";

function mustEnv(name: string) {
  const v = Deno.env.get(name);
  if (!v) throw new Error(`missing_env:${name}`);
  return v;
}

const SUPABASE_URL = mustEnv("SUPABASE_URL");
const SERVICE_ROLE_KEY = mustEnv("SUPABASE_SERVICE_ROLE_KEY");
const APP_ID = "DEBACU_EVAL";

function supabaseServiceClient() {
  return createClient(SUPABASE_URL, SERVICE_ROLE_KEY, {
    auth: { persistSession: false, autoRefreshToken: false },
  });
}

function err(req: Request, status: number, detail: string) {
  return json(req, status, { ok: false, error: "request_failed", detail });
}

async function readJson(req: Request) {
  try {
    const t = await req.text();
    if (!t) return {};
    return JSON.parse(t);
  } catch {
    return {};
  }
}

function safeStr(v: any) {
  return typeof v === "string" ? v.trim() : "";
}

function toYMD(v: any): string | null {
  if (!v) return null;
  const s = String(v);
  return s.length >= 10 ? s.slice(0, 10) : null;
}

function todayYMD(): string {
  return new Date().toISOString().slice(0, 10);
}

/* =====
 * Multi-org resolution
 * - body.org_id recomendado
 * - si no viene: primera membership ACTIVE (determinista)
```

```

* ===== */
async function resolveOrgForUser(
  sb: ReturnType<typeof supabaseServiceClient>,
  userId: string,
  orgId?: string,
) {
  const org_id_in = safeStr(orgId);

  if (org_id_in) {
    const { data, error } = await sb
      .from("debacu_eval_org_members")
      .select("org_id, role, status, created_at")
      .eq("user_id", userId)
      .eq("org_id", org_id_in)
      .eq("status", "ACTIVE")
      .maybeSingle();

    if (error) throw new Error("membership_check_failed");
    if (!data?.org_id) throw new Error("FORBIDDEN");
    return { org_id: String(data.org_id), role: data.role ?? null };
  }

```

```

  const { data, error } = await sb
    .from("debacu_eval_org_members")
    .select("org_id, role, status, created_at")
    .eq("user_id", userId)
    .eq("status", "ACTIVE")
    .order("created_at", { ascending: true })
    .limit(1)
    .maybeSingle();

  if (error) throw new Error("membership_lookup_failed");
  if (!data?.org_id) throw new Error("FORBIDDEN");
  return { org_id: String(data.org_id), role: data.role ?? null };
}

```

```

async function resolveCustomerIdForOrg(sb: ReturnType<typeof supabaseServiceClient>, org_id: string) {
  // 1) entitlements view si existe
  try {
    const { data: ent, error: entErr } = await sb
      .from("debacu_eval_org_entitlements_v")
      .select("customer_id")
      .eq("org_id", org_id)
      .maybeSingle();

    if (!entErr && ent?.customer_id) return String(ent.customer_id);
  } catch {
    // ignore
  }

```

```

  // 2) organizations fallback
  const { data: org, error } = await sb
    .from("debacu_eval_organizations")

```

```

        .select("customer_id")
        .eq("id", org_id)
        .maybeSingle();

    if (error) throw new Error("org_lookup_failed");
    if (!org?.customer_id) throw new Error("FORBIDDEN");
    return String(org.customer_id);
}

async function isAdminUser(sb: ReturnType<typeof supabaseServiceClient>, userId: string) {
    const { data, error } = await sb
        .from("debacu_eval_admin_users")
        .select("user_id")
        .eq("user_id", userId)
        .eq("is_active", true)
        .maybeSingle();

    if (error) return false;
    return !!data?.user_id;
}

/* =====
 * Subscription resolution
 * - ACTIVE/TRIAL_ACTIVE preferente
 * - si no hay: 402 PLAN_NOT_ACTIVE
 * ===== */
async function getActiveSubscription(
    sb: ReturnType<typeof supabaseServiceClient>,
    customer_id: string,
    app_id: string,
) {
    const { data: activeSubs, error } = await sb
        .from("subscriptions")
        .select("id, plan_id, status, start_date, end_date, next_billing_date, billing_frequency, created_at,
updated_at")
        .eq("customer_id", customer_id)
        .eq("app_id", app_id)
        .in("status", ["ACTIVE", "TRIAL_ACTIVE"])
        .order("start_date", { ascending: false })
        .order("updated_at", { ascending: false })
        .order("created_at", { ascending: false })
        .limit(1);

    if (error) throw new Error("db_subscriptions_read_failed");

    if (!activeSubs || activeSubs.length === 0) {
        // Opcional: puedes mirar la última para reportar estado, pero tu estándar pide PLAN_NOT_ACTIVE
        throw new Error("PLAN_NOT_ACTIVE");
    }

    const sub = activeSubs[0] as any;
    const status = String(sub.status ?? "").toUpperCase();

    // Validación extra de trial caducada

```

```

    if (status === "TRIAL_ACTIVE") {
      const endYMD = toYMD(sub.end_date);
      const today = todayYMD();
      if (endYMD && endYMD < today) throw new Error("PLAN_NOT_ACTIVE");
    }

    return sub;
  }

  async function getPlanInfo(sb: ReturnType<typeof supabaseServiceClient>, plan_id: string) {
    const { data, error } = await sb
      .from("plans")
      .select("id, name, code, price_monthly")
      .eq("id", plan_id)
      .maybeSingle();

    if (error || !data) return null;
    return data as any;
  }

  function planTypeFromCode(codeRaw: any) {
    const code = String(codeRaw ?? "").toUpperCase();
    if (!code) return "UNKNOWN";
    if (code === "FREE") return "FREE";
    if (code.includes("BASIC")) return "BASIC";
    if (code.includes("MEDIUM")) return "MEDIUM";
    if (code.includes("PREMIUM")) return "PREMIUM";
    return code;
  }

  /* =====
  * Main
  * ===== */
  Deno.serve(async (req) => {
    if (req.method === "OPTIONS") return preflight(req);
    if (req.method !== "POST") return err(req, 405, "method_not_allowed");

    try {
      // ☒ JWT-only
      const user = await requireUser(req);

      const sb = supabaseServiceClient();
      const body = await readJson(req);

      // multi-org
      const org_id = safeStr(body?.org_id ?? body?.orgId ?? "");
      const { org_id: resolvedOrgId } = await resolveOrgForUser(sb, user.id, org_id || undefined);
      const customer_id = await resolveCustomerIdForOrg(sb, resolvedOrgId);

      // customer (para nombre/email, etc.)
      const { data: customer, error: custErr } = await sb
        .from("customers")
        .select("id, name, email, is_active, start_date")
        .eq("id", customer_id)

```

```
.maybeSingle();
```

```
if (custErr) return err(req, 500, "db_customer_read_failed");  
if (!customer?.id) return err(req, 403, "FORBIDDEN");  
if (customer.is_active === false) return err(req, 403, "FORBIDDEN");
```

```
const adminFlag = await isAdminUser(sb, user.id);
```

```
// subscripción / plan (si admin, no forzamos plan)
```

```
let sub: any = null;  
let planCode: string | null = null;  
let planType = adminFlag ? "ADMIN" : "UNKNOWN";  
let monthlyFee = 0;
```

```
if (!adminFlag) {  
  sub = await getActiveSubscription(sb, customer_id, APP_ID);
```

```
  if (sub?.plan_id) {  
    const plan = await getPlanInfo(sb, String(sub.plan_id));  
    if (plan) {  
      planCode = String(plan.code ?? "").toUpperCase() || null;  
      planType = planTypeFromCode(planCode);  
      monthlyFee = Number(plan.price_monthly ?? 0);  
    }  
  }  
}
```

```
const user_payload = {  
  id: customer_id,  
  customerId: customer_id,  
  orgId: resolvedOrgId,  
  fullName: customer.name ?? "Cliente",  
  email: String(customer.email ?? user.email ?? "").toLowerCase(),  
  plan: planType,  
  planCode,  
  planStartDate: customer.start_date ?? (sub?.start_date ?? ""),  
  monthlyFee,  
  isAdmin: adminFlag,  
  subscriptionStatus: sub?.status ?? (adminFlag ? "ADMIN" : null),  
  billingFrequency: sub?.billing_frequency ?? null,  
  subscriptionId: sub?.id ?? null,  
};
```

```
// ☒ Ya no devolvemos session_token (legacy eliminado)
```

```
return json(req, 200, { ok: true, user: user_payload });  
} catch (e: any) {  
  const msg = String(e?.message ?? e);
```

```
// mapping estricto
```

```
if (msg === "UNAUTHENTICATED" || msg === "UNAUTHORIZED") return err(req, 401, "UNAUTHORIZED");  
if (msg === "FORBIDDEN") return err(req, 403, "FORBIDDEN");  
if (msg === "PLAN_NOT_ACTIVE") return err(req, 402, "PLAN_NOT_ACTIVE");
```

```

    if (msg.startsWith("missing_") || msg.startsWith("invalid_")) return err(req, 400, msg);

    return err(req, 500, "internal_error");
  }
});

=====

// supabase/functions/debacu_eval_auth_postlogin/index.ts
// deno-lint-ignore-file no-explicit-any
import "jsr:@supabase/functions-js/edge-runtime.d.ts";

import { json, preflight } from "../_shared/cors.ts";
import { requireUser, requireAdmin, supabaseServiceClient } from "../_shared/auth.ts";

const DEFAULT_APP_CODE = "DEBACU_EVAL";
const DEFAULT_APP_ID = "DEBACU_EVAL";

type Body = {
  appCode?: string; // "DEBACU_EVAL"
  org_id?: string; // recomendado: UI lo manda
};

type PaywallErrorCode =
  | "UNAUTHENTICATED"
  | "FORBIDDEN"
  | "missing_org_id"
  | "invalid_org_id"
  | "NO_ORG_MEMBERSHIP"
  | "NO_ORG"
  | "NO_ENTITLEMENTS"
  | "PLAN_NOT_ACTIVE"
  | "DATA_INCONSISTENT"
  | "DB_ERROR"
  | "METHOD_NOT_ALLOWED";

function safeStr(v: any) {
  return typeof v === "string" ? v.trim() : "";
}

function normalizeEmail(v: any) {
  return safeStr(v).toLowerCase();
}

async function readJsonSafe<T>(req: Request): Promise<T | null> {
  try {
    const text = await req.text();
    if (!text) return null;
    return JSON.parse(text) as T;
  } catch {
    return null;
  }
}

function fail(req: Request, status: number, code: PaywallErrorCode, extra?: Record<string, unknown>) {
  return json(req, status, {
    ok: false,
    error: "request_failed",
  });
}

```

```

    detail: code,
    ...(extra ?? {}),
  });
}

```

```

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return preflight(req);
  if (req.method !== "POST") return fail(req, 405, "METHOD_NOT_ALLOWED");

```

```

  // 1) Auth (JWT-only)
  let authUser: any;
  try {
    authUser = await requireUser(req);
  } catch {
    return fail(req, 401, "UNAUTHENTICATED");
  }
}

```

```

const authUserId = safeStr(authUser?.id);
const authEmail = normalizeEmail(authUser?.email);

```

```

if (!authUserId) return fail(req, 401, "UNAUTHENTICATED");

```

```

// 2) Body
const body = (await readJsonSafe<Body>(req)) ?? {};
const appCode = safeStr(body.appCode) || DEFAULT_APP_CODE;
const appId = appCode || DEFAULT_APP_ID;
const requestedOrgId = safeStr(body.org_id ?? "");

```

```

// 3) Service client (consistencia + evitar líos de RLS en postlogin)
const sb = supabaseServiceClient();

```

```

/**
 * =====
 * MEMBERSHIP HELPERS
 * =====
 * Regla: CANÓNICO = auth_user_id
 * Soportamos legacy (user_id) SOLO para auto-fix (si existe).
 */

```

```

async function fixLegacyMembershipAuthUserId(orgId: string) {
  // Si existe una fila legacy ACTIVE con user_id==authUserId y auth_user_id null, la reparamos.
  const { data: legacy, error: legacyErr } = await sb
    .from("debacu_eval_org_members")
    .select("id, org_id, status, role, user_id, auth_user_id")
    .eq("org_id", orgId)
    .eq("user_id", authUserId)
    .is("auth_user_id", null)
    .maybeSingle();

```

```

  if (legacyErr) return { ok: false as const, err: legacyErr };
  if (!legacy?.id) return { ok: true as const, changed: false as const };

```

```

  // OJO: si tienes un UNIQUE que impide múltiples ACTIVE por auth_user_id,

```

```

// aquí podría fallar si el usuario ya está ACTIVE en otra org.
const { error: upErr } = await sb
  .from("debacu_eval_org_members")
  .update({
    auth_user_id: authUserId, // ☒ canónico
    updated_at: new Date().toISOString(),
  })
  .eq("id", legacy.id);

if (upErr) return { ok: false as const, err: upErr };
return { ok: true as const, changed: true as const };
}

async function promoteInviteToActive(orgId: string) {
  if (!authEmail) return { ok: false as const, reason: "no_email" as const };

  const { data, error } = await sb
    .from("debacu_eval_org_members")
    .update({
      status: "ACTIVE",
      auth_user_id: authUserId,
      updated_at: new Date().toISOString(),
    })
    .eq("org_id", orgId)
    .eq("status", "INVITED")
    .ilike("invited_email", authEmail)
    .select("org_id, role, status")
    .maybeSingle();

  if (error) return { ok: false as const, reason: "db" as const, err: error };
  if (!data?.org_id) return { ok: false as const, reason: "not_found" as const };
  return { ok: true as const, mem: data };
}

async function loadActiveMembershipInOrg(orgId: string) {
  // Primero intenta canónico
  const { data: mem, error: memErr } = await sb
    .from("debacu_eval_org_members")
    .select("id, org_id, role, status, auth_user_id")
    .eq("org_id", orgId)
    .eq("auth_user_id", authUserId)
    .eq("status", "ACTIVE")
    .maybeSingle();

  if (memErr) return { kind: "error" as const, error: memErr };
  if (mem?.org_id) return { kind: "found" as const, mem };

  // Auto-fix legacy (user_id) si aplica
  const fix = await fixLegacyMembershipAuthUserId(orgId);
  if (!fix.ok) return { kind: "error" as const, error: fix.err };

  if (fix.changed) {

```



```

const { data: mem2, error: mem2Err } = await sb
  .from("debacu_eval_org_members")
  .select("id, org_id, role, status, auth_user_id")
  .eq("org_id", orgId)
  .eq("auth_user_id", authUserId)
  .eq("status", "ACTIVE")
  .maybeSingle();
if (mem2Err) return { kind: "error" as const, error: mem2Err };
if (mem2?.org_id) return { kind: "found" as const, mem: mem2 };
}

// Si no hay ACTIVE, intenta promover invitación por email dentro de ese org
const promoted = await promoteInviteToActive(orgId);
if (promoted.ok) return { kind: "found" as const, mem: promoted.mem };
if ((promoted as any).reason === "db") return { kind: "error" as const, error: (promoted as any).err };

return { kind: "none" as const };
}

```

```

/**
 * =====
 * ORG RESOLUTION (determinista)
 * =====
 * Regla nueva:
 * - Si NO viene org_id, solo resolvemos automáticamente si hay EXACTAMENTE 1 membership ACTIVE.
 * - Si hay >1 -> devolvemos missing_org_id con lista de org_ids para que el frontend elija.
 *
 * Además, soportamos legacy user_id:
 * - Si no hay memberships por auth_user_id, buscamos por user_id (ACTIVE).
 * - Si hay exactamente 1 por user_id, intentamos canonizar (fixLegacyMembershipAuthUserId) y devolvemos.
 * - Si hay >1 por user_id, también es ambiguo.
 */
async function resolveOrgIdFallback() {
  // 1) Canonical: memberships ACTIVE por auth_user_id
  const { data: mems, error: memsErr } = await sb
    .from("debacu_eval_org_members")
    .select("org_id, role, status, created_at")
    .eq("auth_user_id", authUserId)
    .eq("status", "ACTIVE")
    .order("created_at", { ascending: true });

  if (memsErr) return { kind: "error" as const, error: memsErr };

  if (mems && mems.length === 1 && mems[0]?.org_id) {
    return {
      kind: "found" as const,
      orgId: String(mems[0].org_id),
      role: mems[0].role ?? null,
      status: mems[0].status ?? "ACTIVE",
    };
  }
}

```

```

if (mems && mems.length > 1) {
  return {
    kind: "ambiguous" as const,
    orgIds: mems.map((m) => String(m.org_id)),
  };
}

// 2) Legacy: memberships ACTIVE por user_id
const { data: memsLegacy, error: memsLegacyErr } = await sb
  .from("debacu_eval_org_members")
  .select("org_id, role, status, created_at, auth_user_id, user_id")
  .eq("user_id", authUserId)
  .eq("status", "ACTIVE")
  .order("created_at", { ascending: true });

if (memsLegacyErr) return { kind: "error" as const, error: memsLegacyErr };

```

```

if (memsLegacy && memsLegacy.length === 1 && memsLegacy[0]?.org_id) {
  const orgId = String(memsLegacy[0].org_id);

```

```

  // Intento de canonización (si auth_user_id está a null)
  await fixLegacyMembershipAuthUserId(orgId);

```

```

  return {
    kind: "found" as const,
    orgId,
    role: memsLegacy[0].role ?? null,
    status: memsLegacy[0].status ?? "ACTIVE",
  };
}

```

```

if (memsLegacy && memsLegacy.length > 1) {
  return {
    kind: "ambiguous" as const,
    orgIds: memsLegacy.map((m) => String(m.org_id)),
  };
}

```

```

// 3) Si no hay ACTIVE, intenta encontrar una invitación (INVITED) por email y promoverla,
// pero SOLO si hay exactamente 1 invitación (si hay varias también es ambiguo).
if (!authEmail) return { kind: "none" as const };

```

```

const { data: invs, error: invErr } = await sb
  .from("debacu_eval_org_members")
  .select("org_id, created_at")
  .eq("status", "INVITED")
  .ilike("invited_email", authEmail)
  .order("created_at", { ascending: true });

```

```

if (invErr) return { kind: "error" as const, error: invErr };
if (!invs || invs.length === 0) return { kind: "none" as const };

```

```

    if (invs.length > 1) {
      return {
        kind: "ambiguous" as const,
        orgIds: invs.map((i) => String(i.org_id)),
      };
    }

    const onlyOrgId = String(invs[0].org_id);
    const promoted = await promoteInviteToActive(onlyOrgId);
    if (!promoted.ok) return { kind: "none" as const };

    return { kind: "found" as const, orgId: String(promoted.mem.org_id), role: promoted.mem.role, status:
promoted.mem.status };
  }

  /**
   * =====
   * 4) Resolver org_id + validar membership ACTIVE
   * =====
   */
  let orgId: string | null = requestedOrgId || null;
  let memberRole: string | null = null;
  let memberStatus: string | null = null;

  if (orgId) {
    const res = await loadActiveMembershipInOrg(orgId);
    if (res.kind === "error") return fail(req, 500, "DB_ERROR");
    if (res.kind === "none") return fail(req, 403, "NO_ORG_MEMBERSHIP", { org_id: orgId });

    memberRole = res.mem.role ?? null;
    memberStatus = res.mem.status ?? "ACTIVE";
  } else {
    const res = await resolveOrgIdFallback();
    if (res.kind === "error") return fail(req, 500, "DB_ERROR");
    if (res.kind === "ambiguous") {
      // frontend debe mandar org_id explícito
      return fail(req, 400, "missing_org_id", { org_ids: res.orgIds });
    }
    if (res.kind === "none") return fail(req, 403, "NO_ORG_MEMBERSHIP");
  }

  orgId = res.orgId;
  memberRole = res.role ?? null;
  memberStatus = res.status ?? "ACTIVE";
}

  /**
   * =====
   * 5) org -> customer_id
   * =====
   */
  const { data: org, error: orgErr } = await sb
    .from("debacu_eval_organizations")
    .select("id, customer_id")

```

```

        .eq("id", orgId)
        .maybeSingle();

    if (orgErr) return fail(req, 500, "DB_ERROR");
    if (!org?.customer_id) return fail(req, 403, "NO_ORG", { org_id: orgId });

```

```

const customerId = String(org.customer_id);

```

```

/**
 * =====
 * 6) Entitlements (view)
 * =====
 */

```

```

const { data: ent, error: entErr } = await sb
    .from("debacu_eval_org_entitlements_v")
    .select(
        [
            "org_id",
            "customer_id",
            "seats_used",
            "plan_code",
            "max_users",
            "extra_seats",
            "seats_total",
            "seats_available",
            "subscription_status",
        ].join(","),
    )
    .eq("org_id", orgId)
    .maybeSingle();

```

```

if (entErr) return fail(req, 500, "DB_ERROR");

```

```

// Consistencia: ent.customer_id debe coincidir con org.customer_id
if (ent?.customer_id && String(ent.customer_id) !== customerId) {
    return fail(req, 500, "DATA_INCONSISTENT", {
        appCode,
        appId,
        org_id: orgId,
        orgCustomerId: customerId,
        entitlementCustomerId: ent.customer_id,
    });
}

```

```

/**
 * =====
 * 7) Admin bypass (tabla debacu_eval_admin_users via requireAdmin)
 * =====
 */
let isPlatformAdmin = false;
try {
    await requireAdmin(req);

```

```

    isPlatformAdmin = true;
} catch {
    isPlatformAdmin = false;
}

```

```

if (isPlatformAdmin) {
    return json(req, 200, {
        ok: true,
        data: {
            user: { id: authUserId, email: authEmail || null },
            customer: { id: customerId },
            membership: { org_id: orgId, role: "PLATFORM_ADMIN", status: "ACTIVE", base_role: memberRole },
            entitlement: {
                customer_id: customerId,
                plan_code: "INTERNAL_ADMIN",
                subscription_status: "ACTIVE",
                seats_used: Number(ent?.seats_used ?? 1),
                max_users: 999999,
                extra_seats: Number((ent as any)?.extra_seats ?? 0),
                seats_total: Number((ent as any)?.seats_total ?? 999999),
                seats_available: Number((ent as any)?.seats_available ?? 999999),
            },
        },
    });
}

```

```

/**
 * =====
 * 8) Normal users: requiere entitlement activo
 * =====
 */
if (!ent?.subscription_status || !ent?.plan_code) {
    return fail(req, 402, "PLAN_NOT_ACTIVE", {
        appCode,
        appId,
        org_id: orgId,
        customer_id: customerId,
        subscription_status: ent?.subscription_status ?? null,
        plan_code: ent?.plan_code ?? null,
    });
}

```

```

const status = String(ent.subscription_status);
const entitled = status === "ACTIVE" || status === "TRIAL_ACTIVE";

```

```

if (!entitled) {
    return fail(req, 402, "PLAN_NOT_ACTIVE", {
        appCode,
        appId,
        org_id: orgId,
        customer_id: customerId,
        subscription_status: status,
    });
}

```

```

});
}

return json(req, 200, {
  ok: true,
  data: {
    user: { id: authUserId, email: authEmail || null },
    customer: { id: customerId },
    membership: { org_id: orgId, role: memberRole ?? "USER", status: memberStatus ?? "ACTIVE" },
    entitlement: {
      customer_id: customerId,
      plan_code: ent.plan_code,
      subscription_status: status,
      seats_used: Number(ent.seats_used ?? 0),
      max_users: Number(ent.max_users ?? 0),
      extra_seats: Number((ent as any).extra_seats ?? 0),
      seats_total: Number((ent as any).seats_total ?? 0),
      seats_available: Number((ent as any).seats_available ?? 0),
    },
  },
});
});

```

AL IR A SOLICITAR ACCESO Y RELLENAR FORMULARIO, ACEPTAR CONDICIONES

The screenshot displays a web application interface for 'Debacu Evaluation360'. The main content area shows the 'Términos y condiciones' (Terms and Conditions) page, which includes an 'Aviso Legal' (Legal Notice) section. The sidebar on the left contains a 'DOCUMENTOS' (Documents) section with a 'Términos' (Terms) link. The right sidebar features buttons for 'Aceptar términos' (Accept terms) and 'Cerrar' (Close). The bottom of the page indicates the last update was on 24-01-2026 and provides a contact email for legal matters: legal@debacu.com.

FUNCION

```

import "jsr:@supabase/functions-js/edge-runtime.d.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.45.4";

const corsHeaders = {

```

```

"Access-Control-Allow-Origin": "*",
"Access-Control-Allow-Headers":
  "authorization, x-client-info, apikey, content-type",
"Access-Control-Allow-Methods": "POST, OPTIONS",
};

function json(status: number, body: unknown) {
  return new Response(JSON.stringify(body), {
    status,
    headers: { ...corsHeaders, "Content-Type": "application/json" },
  });
}

function requireEnv(name: string) {
  const v = Deno.env.get(name);
  if (!v) throw new Error(`Missing env: ${name}`);
  return v;
}

type PropertyType = "HOTEL" | "RURAL" | "APARTMENTS" | "HOSTEL" | "OTHER";

type Body = {
  company_name: string;
  legal_name?: string;
  cif: string;
  address?: string;
  city?: string;
  country?: string;
  property_type: PropertyType;
  rooms_count?: number;
  website?: string;
  contact_name: string;
  contact_role?: string;
  email: string;
  phone?: string;
  notes?: string;
  accepted_professional_use?: boolean;
};

Deno.serve(async (req) => {
  try {
    if (req.method === "OPTIONS") {
      return new Response("ok", { status: 200, headers: corsHeaders });
    }
    if (req.method !== "POST") return json(405, { error: "Method not allowed" });

    const SUPABASE_URL = requireEnv("SUPABASE_URL");
    const SERVICE_ROLE_KEY = requireEnv("SUPABASE_SERVICE_ROLE_KEY");
    const supabase = createClient(SUPABASE_URL, SERVICE_ROLE_KEY);

    const body = (await req.json()) as Body;

    const company_name = body.company_name?.trim();
    const legal_name = body.legal_name?.trim() || null;

```

```

const cif = body.cif?.trim();
const address = body.address?.trim() || null;
const city = body.city?.trim() || null;
const country = body.country?.trim() || "ESP";
const property_type = body.property_type;
const website = body.website?.trim() || null;
const contact_name = body.contact_name?.trim();
const contact_role = body.contact_role?.trim() || null;
const email = body.email?.trim();
const phone = body.phone?.trim() || null;
const notes = body.notes?.trim() || null;

if (!company_name) return json(400, { error: "company_name required" });
if (!cif) return json(400, { error: "cif required" });
if (!contact_name) return json(400, { error: "contact_name required" });
if (!email || !email.includes("@")) return json(400, { error: "valid email required" });
if (!property_type) return json(400, { error: "property_type required" });

const rooms_count =
  typeof body.rooms_count === "number" && Number.isFinite(body.rooms_count)
    ? body.rooms_count
    : null;

// ☒ evitar duplicados de PENDING (opcional)
const { data: existing, error: existingErr } = await supabase
  .from("debacu_eval_access_requests")
  .select("id,status,accepted_terms,created_at")
  .eq("status", "PENDING")
  .eq("email", email)
  .eq("cif", cif)
  .order("created_at", { ascending: false })
  .limit(1)
  .maybeSingle();

if (existingErr) return json(500, { error: existingErr.message });
if (existing?.id) {
  return json(200, {
    id: existing.id,
    status: existing.status,
    duplicate: true,
    accepted_terms: existing.accepted_terms ?? false,
  });
}

const { data, error } = await supabase
  .from("debacu_eval_access_requests")
  .insert({
    status: "PENDING",
    company_name,
    legal_name,
    cif,
    address,
  })

```



```

city,
country,
property_type,
rooms_count,
website,
contact_name,
contact_role,
email,
phone,
notes,

```

```

accepted_terms: false,
accepted_professional_use: !!body.accepted_professional_use,
})
.select("id,status,accepted_terms")
.maybeSingle();

```

```

if (error) return json(500, { error: error.message });
if (!data?.id) return json(500, { error: "pending request not created" });

```

```

return json(200, { id: data.id, status: data.status, accepted_terms: data.accepted_terms });
} catch (e: any) {
return json(500, { error: e?.message ?? String(e) });
}
});

```

ACEPTAMOS CONDICIONES

The screenshot shows a web application interface for accepting terms and conditions. The form includes fields for 'Pais' (ESP), 'Nº habitaciones' (10), and 'Web' (WWW.HOTEL2.COM). A section titled 'Declaraciones' contains a 'Términos y condiciones' box with a 'Ver y aceptar' button, and a checkbox for 'Declaro uso profesional y responsable de los datos'. A green button at the bottom says 'Aceptado. Justificante generado y guardado.'.

Overlaid on the right is a network developer tool showing a list of API calls. The table below represents the data from this tool:

Name	Status	Type	Initiator	Size	Time	Fulfill...
debacu_eval_properties?select=id%2Corg_id%2...	200	fetch	fetch.ts:7	1.2 kB	204 ms	
debacu_eval_dashboard_revenue_month	200	fetch	callEvalFn.ts:78	1.1 kB	557 ms	
debacu_eval_dashboard_revenue_month	200	fetch	callEvalFn.ts:78	1.1 kB	557 ms	
client_dashboard	200	fetch	clientService.ts:84	1.6 kB	732 ms	
client_dashboard	200	fetch	clientService.ts:84	1.6 kB	729 ms	
logout?scope=global	200	preflight	Preflight@	0 B	23 ms	
logout?scope=global	204	fetch	helpers.ts:109	593 B	78 ms	
debacuevaluation.png	200	png	react-dom-client.dev	2.3 MB	378 ms	
hotel.png	200	png	Other	3.1 MB	1.01 s	
rural.png	200	png	Other	312 kB	450 ms	
apartments.png	200	png	Other	312 kB	464 ms	
hostel.png	200	png	Other	316 kB	481 ms	
reception.png	200	png	Other	2.2 MB	732 ms	
director.png	200	png	Other	2.2 MB	444 ms	
operations.png	200	png	Other	2.2 MB	465 ms	
frontdesk.png	200	png	Other	2.1 MB	395 ms	
audit.png	200	png	Other	206 kB	99 ms	
sales.png	200	png	Other	1.9 MB	341 ms	
cartel1.png	200	png	react-dom-client.dev	2.4 MB	60 ms	
data:image/svg+xml...	200	svg+xml	Other	0 B	0 ms	(me...
debacu_eval_access_request_create_draft	200	preflight	Preflight@	0 B	225 ms	
debacu_eval_access_request_create_draft	200	fetch	debacu_eval_accessRe	746 B	412 ms	
debacu_eval_generate_terms_acceptance	200	fetch	debacu_eval_accessRe	1.2 kB	1.04 s	
debacu_eval_generate_terms_acceptance	204	preflight	Preflight@	0 B	339 ms	

Summary: 202 requests, 33.2 MB transferred, 33.2 MB resources, Finish: 2.8 min, DOMContentLoaded: 1.82 s, Load: 1.83 s.

```

FUNCION // supabase/functions/debacu_eval_generate_terms_acceptance/index.ts
// deno-lint-ignore-file no-explicit-any
import "jsr:@supabase/functions-js/edge-runtime.d.ts";

```

```

import { PDFDocument, StandardFonts, rgb } from "https://esm.sh/pdf-lib@1.17.1";

import { json, preflight } from "../_shared/cors.ts";
import { supabaseServiceClient } from "../_shared/auth.ts";

/**
 * ☒ Server-side versioning (NO confiar en frontend)
 */
const TERMS_VERSION = "2026-01-24 - V1.0";

/**
 * Storage bucket destino
 */
const LEGAL_BUCKET = "debacu_legal_acceptances";

type PropertyType = "HOTEL" | "RURAL" | "APARTMENTS" | "HOSTEL" | "OTHER";

type Body = {
  request_id?: string;
  /**
   * Recomendado: envíalo desde el frontend para validar que quien acepta
   * es el mismo email de la solicitud.
   */
  email?: string;
};

type PdfData = {
  request_id: string;
  terms_version: string;

  company_name: string;
  cif: string;
  contact_name: string;
  email: string;

  property_type?: PropertyType | null;
  city?: string | null;
  country?: string | null;
  legal_name?: string | null;
  address?: string | null;
  rooms_count?: number | null;
  website?: string | null;
  contact_role?: string | null;
  phone?: string | null;
  notes?: string | null;

  accepted_ip?: string | null;
  accepted_user_agent?: string | null;

  // ☒ Anexos de seguridad/RGPD (texto controlado server-side)
  // (No son datos personales del solicitante; son cláusulas del documento)
  rgpd_retention_years?: number | null;
  rgpd_log_retention_days?: number | null;
};

```

```

function toHex(buffer: ArrayBuffer) {
  return [...new Uint8Array(buffer)].map((b) => b.toString(16).padStart(2, "0")).join("");
}

function nowIso() {
  return new Date().toISOString();
}

function safeTimestamp(iso: string) {
  // 2026-02-18T08:10:00.123Z -> 2026-02-18T08-10-00-123Z
  return iso.replace(/[:.]/g, "-");
}

function getClientIp(req: Request): string | null {
  const xff = req.headers.get("x-forwarded-for");
  if (xff) return xff.split(",")[0]?.trim() ?? null;
  return req.headers.get("cf-connecting-ip") ?? null;
}

function normalizeEmail(e: string | null | undefined) {
  return (e ?? "").trim().toLowerCase();
}

function errPayload(code: string, detail?: string) {
  return { ok: false, error: "request_failed", detail: detail ? `${code}:${detail}` : code };
}

function badRequest(req: Request, code: string, detail?: string) {
  return json(req, 400, errPayload(code, detail));
}

function forbidden(req: Request, code = "forbidden") {
  return json(req, 403, errPayload(code));
}

function notFound(req: Request) {
  return json(req, 404, errPayload("not_found"));
}

function serverError(req: Request, code: string, detail?: string) {
  return json(req, 500, errPayload(code, detail));
}

/**
 * PDF generator (igual que el tuyo)
 */
async function buildPdf(data: PdfData, acceptedAtIso: string) {
  const pdfDoc = await PDFDocument.create();
  const font = await pdfDoc.embedFont(StandardFonts.Helvetica);
  const fontBold = await pdfDoc.embedFont(StandardFonts.HelveticaBold);

  const PAGE_W = 595.28; // A4
  const PAGE_H = 841.89; // A4
  const margin = 48;
  const contentW = PAGE_W - margin * 2;

  const colors = {

```

```
title: rgb(0.05, 0.1, 0.2),
text: rgb(0.2, 0.2, 0.2),
muted: rgb(0.45, 0.45, 0.45),
subtle: rgb(0.75, 0.78, 0.82),
};
```

```
let page = pdfDoc.addPage([PAGE_W, PAGE_H]);
let y = PAGE_H - margin;
let pageNo = 1;
```

```
const footer = () => {
  const footerY = 24;
  page.drawText(`Debacu Evaluation360 · Centro legal`, {
    x: margin,
    y: footerY,
    size: 9,
    font,
    color: colors.muted,
  });
  page.drawText(`Página ${pageNo}`, {
    x: PAGE_W - margin - 60,
    y: footerY,
    size: 9,
    font,
    color: colors.muted,
  });
};
```

```
const newPage = () => {
  footer();
  page = pdfDoc.addPage([PAGE_W, PAGE_H]);
  pageNo += 1;
  y = PAGE_H - margin;
};
```

```
const ensureSpace = (need: number) => {
  const bottom = 48;
  if (y - need < bottom) newPage();
};
```

```
const hr = () => {
  ensureSpace(16);
  y -= 6;
  page.drawLine({
    start: { x: margin, y },
    end: { x: PAGE_W - margin, y },
    thickness: 1,
    color: colors.subtle,
  });
  y -= 10;
};
```

```

const wrapLines = (text: string, size: number) => {
  const words = text.replace(/\s+/g, " ").trim().split(" ").filter(Boolean);
  const lines: string[] = [];
  let line = "";
  for (const w of words) {
    const test = line ? `${line} ${w}` : w;
    const width = font.widthOfTextAtSize(test, size);
    if (width > contentW) {
      if (line) lines.push(line);
      line = w;
    } else {
      line = test;
    }
  }
  if (line) lines.push(line);
  return lines;
};

```

```

const drawH1 = (text: string) => {
  ensureSpace(40);
  page.drawText(text, { x: margin, y, size: 18, font: fontBold, color: colors.title });
  y -= 28;
};

```

```

const drawH2 = (text: string) => {
  ensureSpace(26);
  page.drawText(text, { x: margin, y, size: 12.5, font: fontBold, color: rgb(0.12, 0.12, 0.12) });
  y -= 18;
};

```

```

const drawP = (text: string, size = 10.5, lineGap = 14.5) => {
  const lines = wrapLines(text, size);
  ensureSpace(lines.length * lineGap + 10);
  for (const l of lines) {
    page.drawText(l, { x: margin, y, size, font, color: colors.text });
    y -= lineGap;
  }
  y -= 6;
};

```

```

const drawList = (items: string[], bullet = "•") => {
  const size = 10.5;
  const lineGap = 14.5;
  for (const it of items) {
    const lines = wrapLines(it, size);
    ensureSpace(lines.length * lineGap + 10);

    page.drawText(bullet, { x: margin, y, size, font, color: colors.text });
    page.drawText(lines[0], { x: margin + 14, y, size, font, color: colors.text });
    y -= lineGap;
  }
}

```

```

  for (let i = 1; i < lines.length; i++) {

```

```

    page.drawText(lines[i], { x: margin + 14, y, size, font, color: colors.text });
    y -= lineGap;
  }
  y -= 2;
}
y -= 6;
};

```

```

const drawKV = (label: string, value: string) => {
  const size = 10.5;
  const labelW = 165;
  ensureSpace(18);

```

```

  page.drawText(label, { x: margin, y, size, font: fontBold, color: rgb(0.1, 0.1, 0.1) });

```

```

  const maxW = contentW - labelW;
  const words = (value || "-").replace(/\s+/g, " ").trim().split(" ");
  let line = "";
  const lines: string[] = [];
  for (const w of words) {
    const test = line ? `${line} ${w}` : w;
    const width = font.widthOfTextAtSize(test, size);
    if (width > maxW) {
      if (line) lines.push(line);
      line = w;
    } else {
      line = test;
    }
  }
  if (line) lines.push(line);

```

```

  page.drawText(lines[0] ?? "-", { x: margin + labelW, y, size, font, color: rgb(0.15, 0.15, 0.15) });
  y -= 14.5;

```

```

  for (let i = 1; i < lines.length; i++) {
    ensureSpace(14.5);
    page.drawText(lines[i], { x: margin + labelW, y, size, font, color: rgb(0.15, 0.15, 0.15) });
    y -= 14.5;
  }

```

```

  y -= 4;
};

```

```

const org = {
  provider_name: "DEBACU HOTELS SL",
  provider_cif: "B-55381214",
  provider_address: "C/CANTALEJO,13-1º A",
  provider_email: "informacion@debacu.com",
  privacy_email: "privacidad@debacu.com",
};

```

```

// ☒ Parámetros server-side recomendados (ajustables)

```

```

const retentionYears = Number.isFinite(Number(data.rgpd_retention_years)) ? Number(data.rgpd_retention_years)
: 3; // tu criterio: 3 años
const logRetentionDays = Number.isFinite(Number(data.rgpd_log_retention_days))
? Number(data.rgpd_log_retention_days)
: 180;

drawH1("Justificante de aceptación · Debacu Evaluation360");
drawP(
  "Este documento reúne en un único PDF la evidencia de aceptación electrónica y el contenido legal aplicable
al acceso y uso profesional de la plataforma (Aviso Legal, Términos y Condiciones, Política de Acceso y Uso
Profesional y Encargo de Tratamiento – RGPD).",
);
hr();

drawH2("Datos de la solicitud");
drawKV("Solicitud (ID):", data.request_id);
drawKV("Fecha/hora aceptación:", acceptedAtIso);
drawKV("Versión documentos:", data.terms_version);

if (data.accepted_ip || data.accepted_user_agent) {
  y -= 2;
  drawH2("Evidencia técnica de aceptación");
  drawKV("IP (x-forwarded-for):", data.accepted_ip ?? "-");
  drawKV("User-Agent:", data.accepted_user_agent ?? "-");
} else {
  drawP(
    "Nota técnica: la IP y el User-Agent se registran a nivel de servidor y quedan asociados a esta solicitud
(ID) en los registros internos.",
  );
}

hr();

drawH2("Datos declarados por el solicitante");
drawKV("Nombre comercial:", data.company_name);
drawKV("Razón social:", data.legal_name ?? "-");
drawKV("CIF:", data.cif);
drawKV("Dirección:", data.address ?? "-");
drawKV("Ciudad:", data.city ?? "-");
drawKV("País:", data.country ?? "-");
drawKV("Tipo alojamiento:", data.property_type ?? "-");
drawKV("Nº habitaciones:", data.rooms_count?.toString() ?? "-");
drawKV("Web:", data.website ?? "-");
drawKV("Responsable:", data.contact_name);
drawKV("Cargo:", data.contact_role ?? "-");
drawKV("Email:", data.email);
drawKV("Teléfono:", data.phone ?? "-");

if (data.notes?.trim()) {
  hr();
  drawH2("Observaciones (aportadas por el solicitante)");
  drawP(data.notes.trim());
}

```

```
hr();
drawH2("Declaración de aceptación electrónica");
drawP(
    "El solicitante declara haber leído y aceptado expresamente los documentos incluidos en este PDF y consiente su incorporación como evidencia de aceptación vinculada a la solicitud indicada.",
);
drawP(
    "Este documento ha sido aceptado electrónicamente, sin necesidad de firma manuscrita, conforme a la Ley 34/2002 (LSSI-CE) y el Reglamento (UE) 910/2014 (eIDAS).",
);

newPage();

drawH1("Documento 1 · Aviso Legal");
drawP(
    "En cumplimiento de la normativa aplicable, se informa que el sitio y la plataforma Debacu Evaluation360 (en adelante, la "Plataforma") es titularidad del proveedor indicado a continuación.",
);
drawH2("1. Titularidad");
drawList([
    `Titular / Razón social: ${org.provider_name}`,
    `CIF/NIF: ${org.provider_cif}`,
    `Domicilio: ${org.provider_address}`,
    `Email de contacto: ${org.provider_email}`,
]);
drawH2("2. Objeto y naturaleza del servicio");
drawP(
    "La Plataforma proporciona un entorno privado de uso profesional para alojamientos y equipos operativos, orientado a la gestión interna de incidencias y trazabilidad (consultas, registros y auditoría). No se trata de un servicio público ni de un registro accesible al público general.",
);
drawH2("3. Acceso y registro");
drawP(
    "El acceso puede requerir alta controlada, creación de cuenta y autenticación. El usuario se compromete a facilitar información veraz, mantenerla actualizada y custodiar sus credenciales, evitando el uso compartido no autorizado.",
);
drawH2("4. Normas de uso");
drawP(
    "El usuario se compromete a utilizar la Plataforma de forma diligente, lícita y conforme a la finalidad profesional descrita. Queda prohibida la difusión pública de información obtenida en la Plataforma, la recolección automatizada no autorizada, así como cualquier uso difamatorio, discriminatorio o contrario a la buena fe.",
);
drawH2("5. Propiedad intelectual e industrial");
drawP(
    "Los contenidos, marcas, diseños, software y elementos de la Plataforma están protegidos por derechos de propiedad intelectual e industrial. Queda prohibida su reproducción, distribución o explotación no autorizada.",
);
drawH2("6. Responsabilidad");
drawP(
    "La Plataforma se ofrece "tal cual", con esfuerzos razonables de disponibilidad y seguridad. El usuario es responsable del uso que haga de la información en su propia operativa.",
);
```



```

drawH2("7. Enlaces y terceros");
drawP("Pueden existir enlaces a sitios de terceros. El titular no se responsabiliza de sus contenidos,
disponibilidad o políticas.");
drawH2("8. Legislación y jurisdicción");
drawP(
    "Este Aviso Legal se rige por la legislación española. Para cualquier controversia, las partes se someterán a
los juzgados y tribunales competentes conforme a la normativa aplicable.",
);

newPage();

drawH1("Documento 2 · Términos y Condiciones");
drawP(
    "Estos términos regulan el acceso y uso de la Plataforma Debacu Evaluation360. Al registrarte, solicitar
acceso o utilizar la Plataforma, aceptas estas condiciones en la versión indicada.",
);
drawH2("1. Definiciones");
drawList([
    "Plataforma: software y servicios Debacu Evaluation360, de acceso privado y uso profesional.",
    "Cliente / Organización: entidad (p. ej., hotel o alojamiento) que solicita acceso y contrata (si aplica)
planes de suscripción.",
    "Usuario: persona física autorizada por la Organización para acceder.",
]);
drawH2("2. Cuenta, acceso y seguridad");
drawList([
    "El usuario es responsable de mantener la confidencialidad de sus credenciales y del uso bajo su cuenta.",
    "La Organización deberá asegurar que solo personal autorizado accede a la Plataforma.",
    "Podemos suspender accesos ante indicios razonables de abuso, fraude, incidentes de seguridad o
incumplimiento.",
]);
drawH2("3. Uso profesional y limitaciones");
drawP(
    "La Plataforma es privada y de uso profesional. Queda prohibida la difusión pública de información, la
extracción masiva o automatizada no autorizada y cualquier uso contrario a la finalidad operativa interna.",
);
drawH2("4. Planes, suscripción y facturación (si aplica)");
drawList([
    "La Plataforma puede ofrecer planes (incluyendo un plan inicial gratuito limitado) y planes de pago.",
    "La facturación y cobros pueden gestionarse mediante proveedores como Stripe u otros equivalentes.",
    "No almacenamos datos completos de tarjeta; el pago se procesa por el proveedor de pagos.",
    "Las condiciones económicas, límites de uso y prestaciones se detallan en la configuración del plan
vigente.",
]);
drawH2("5. Renovación, cambios y cancelación (si aplica)");
drawList([
    "Las suscripciones pueden renovarse automáticamente según el plan y la periodicidad contratada.",
    "El usuario/cliente puede solicitar cambios de plan según disponibilidad y reglas internas de la
Plataforma.",
    "La cancelación puede realizarse desde el área de cuenta o el portal del proveedor de pagos si está
habilitado.",
]);
drawH2("6. Contenidos y responsabilidad del Cliente");
drawP(

```

```

    "El Cliente es responsable de los datos y contenidos que registra en la Plataforma, incluyendo su exactitud,
    pertinencia y adecuación legal. Se recomienda evitar datos excesivos o no pertinentes, así como expresiones
    ofensivas o valoraciones discriminatorias.",
    );
    drawH2("7. Limitación de responsabilidad");
    drawP(
        "La Plataforma ofrece herramientas de apoyo a procesos internos. Las decisiones que el Cliente adopte basadas
        en la información o en su uso operativo son responsabilidad del Cliente. No se garantiza ausencia total de
        errores, interrupciones o indisponibilidades, sin perjuicio de los esfuerzos razonables de continuidad y
        seguridad.",
    );
    drawH2("8. Soporte");
    drawP(
        "El soporte puede variar según el plan. El alcance y tiempos de respuesta podrán definirse en el plan
        contratado o en acuerdos de nivel de servicio (SLA) cuando existan.",
    );
    drawH2("9. Modificaciones");
    drawP(
        "Podemos actualizar estas condiciones por cambios legales o del servicio. La versión vigente estará publicada
        y se identificará por su versión/fecha.",
    );

    newPage();

    drawH1("Documento 3 · Política de Acceso y Uso Profesional");
    drawP(
        "Esta política concreta el carácter restringido, interno y profesional de la Plataforma y establece reglas de
        uso para garantizar trazabilidad, seguridad y calidad de la información.",
    );
    drawH2("1. Acceso restringido");
    drawP(
        "Debacu Evaluation360 es una plataforma privada destinada a profesionales del sector alojamiento. El acceso
        se concede de forma controlada a organizaciones verificadas y usuarios autorizados.",
    );
    drawH2("2. Uso interno y no público");
    drawList([
        "No es un registro público, no es indexable y no está pensado para difusión externa.",
        "La información está orientada a protocolos internos y mejora operativa.",
        "Se prohíbe publicar, compartir o redistribuir contenidos fuera de la organización o sin base legal.",
    ]);
    drawH2("3. Criterios estructurados y minimización");
    drawP(
        "El sistema fomenta el registro estructurado (motivos, tipologías, severidad, fechas y evidencias internas),
        minimizando opiniones y evitando datos excesivos. El Cliente se compromete a registrar solo información
        pertinente, verificable y relacionada con su operativa.",
    );
    drawH2("4. Auditoría y trazabilidad");
    drawP(
        "Para control interno y seguridad, se registran acciones relevantes (consultas, altas, modificaciones,
        cambios de permisos, exportaciones cuando existan) asociadas a la cuenta. Estos registros se usan para prevenir
        abuso, investigar incidencias y reforzar la trazabilidad.",
    );
    drawH2("5. Prohibiciones específicas");
    drawList([
        "Uso discriminatorio o contrario a derechos fundamentales.",
    ]);

```

```

    "Uso como "lista pública" o exposición de terceros.",
    "Extracción masiva o automatizada no autorizada.",
    "Introducir datos sensibles innecesarios o no pertinentes (salvo estricta necesidad y base legal).",
]);
drawH2("6. Medidas ante abuso");
drawP(
    "En caso de uso indebido, el titular podrá suspender o cancelar accesos y/o limitar funcionalidades para
proteger la Plataforma, sin perjuicio de las acciones legales que procedan.",
);

newPage();

drawH1("Documento 4 · Encargo de Tratamiento (DPA) · RGPD");
drawP(
    "Este documento regula el encargo de tratamiento cuando el Cliente incorpora datos personales a la
Plataforma. En un entorno B2B, normalmente el Cliente (hotel/alojamiento) actúa como Responsable del tratamiento
y el proveedor de la Plataforma como Encargado del tratamiento, en los términos del art. 28 RGPD.",
);
drawH2("1. Partes");
drawList([
    `Responsable (Cliente): ${data.legal_name ?? data.company_name} · CIF ${data.cif} · ${data.address ?? "-"} ·
${data.city ?? "-"} · ${
    data.country ?? "-"
}`,
    `Encargado (Proveedor): DEBACU HOTELS SL · CIF B-55381214 · C/CANTALEJO,13-1º A · informacion@debacu.com`,
]);
drawH2("2. Objeto del encargo");
drawP(
    "Prestación del servicio de plataforma privada para gestión operativa con trazabilidad, conforme a
instrucciones documentadas del Responsable, incluyendo: almacenamiento, consulta, registro, modificación,
auditoría y soporte.",
);

// ☒ Añadido: finalidad + prevención de fraude contractual (interés legítimo)
drawH2("3. Finalidad y base jurídica (Responsable)");
drawP(
    "La finalidad del tratamiento por parte del Responsable incluye la gestión operativa y la prevención de
fraude contractual, impagos, daños, no-shows y otras incidencias vinculadas a la relación de hospedaje. La base
jurídica aplicable será la que determine el Responsable (p. ej., ejecución de contrato o interés legítimo del
art. 6.1.f RGPD), cumpliendo el deber de información y, cuando proceda, realizando la ponderación del interés
legítimo.",
);

drawH2("4. Duración");
drawP(
    "Durante la vigencia de la relación contractual o de acceso autorizado al servicio, y mientras sea necesario
para la prestación del mismo, sin perjuicio de obligaciones legales de conservación.",
);

drawH2("5. Naturaleza, categorías de interesados y tipos de datos");
drawList([
    "Naturaleza del tratamiento: recogida por el Responsable, almacenamiento, estructuración, consulta y
auditoría.",
    "Interesados: clientes/huéspedes u otras personas relacionadas con la operativa del Responsable, según el uso
del Responsable.",
]);

```

```
"Tipos de datos: según el uso del Responsable. Se recomienda minimización y estructuración (tipologías,
fechas, importes, evidencias internas).",
]);
drawP(
    "El Responsable se compromete a evitar el registro de datos excesivos, especialmente categorías especiales
(art. 9 RGPD) salvo estricta necesidad, base jurídica y garantías adecuadas.",
);

drawH2("6. Seudonimización y diseño seguro (recomendado)");
drawList([
    "La Plataforma puede emplear identificadores técnicos seudonimizados (p. ej., claves derivadas tipo HMAC)
para evitar el tratamiento innecesario de identificadores directos.",
    "Las claves secretas/pepper necesarias para derivaciones se mantienen fuera del alcance del Cliente y no se
exponen en interfaces públicas.",
    "El Responsable seguirá siendo responsable de la licitud del tratamiento y de no introducir datos directos
innecesarios.",
]);

drawH2("7. Obligaciones del Encargado (art. 28 RGPD)");
drawList([
    "Tratar los datos personales únicamente siguiendo instrucciones documentadas del Responsable, salvo
obligación legal aplicable.",
    "Garantizar que el personal autorizado se compromete a confidencialidad.",
    "Adoptar medidas técnicas y organizativas apropiadas para garantizar un nivel de seguridad adecuado (art. 32
RGPD).",
    "Asistir al Responsable, cuando proceda, en la atención de solicitudes de derechos de los interesados (arts.
12-22 RGPD).",
    "Asistir al Responsable en la gestión de violaciones de seguridad (arts. 33-34 RGPD), sin dilación indebida,
una vez tenga conocimiento.",
    "Poner a disposición del Responsable la información necesaria para demostrar el cumplimiento del art. 28
RGPD, y permitir auditorías razonables (con preaviso y sin comprometer seguridad/terceros).",
]);

drawH2("8. Obligaciones del Responsable");
drawList([
    "Garantizar base jurídica para el tratamiento y deber de información a los interesados cuando proceda.",
    "Configurar y usar la Plataforma conforme a minimización y finalidad operativa.",
    "Gestionar permisos/roles y accesos de sus usuarios autorizados.",
    "Atender solicitudes de derechos y reclamaciones, con apoyo del Encargado cuando aplique.",
]);

drawH2("9. Subencargados");
drawP(
    "El Encargado podrá utilizar subencargados necesarios para la prestación del servicio (p. ej.,
infraestructura/hosting, correo transaccional, pasarela de pagos), garantizando obligaciones equivalentes
mediante acuerdos adecuados.",
);
drawList([
    "Infraestructura/hosting (p. ej., Supabase/Cloud, según configuración).",
    "Correo transaccional (p. ej., Brevo), si se utiliza.",
    "Pagos/suscripciones (p. ej., Stripe), si aplica.",
]);

drawH2("10. Transferencias internacionales");
drawP(
```

```
"Si algún proveedor tratase datos fuera del EEE, se aplicarán garantías adecuadas (p. ej., Cláusulas Contractuales Tipo) y/o decisiones de adecuación, según corresponda.",
);
```

```
// ☒ Añadido: conservación / retención (la pieza que suele faltar)
drawH2("11. Conservación, supresión y limitación temporal");
drawList([
    `El Responsable definirá y aplicará su política de conservación. A efectos orientativos, se recomienda una conservación máxima de ${retentionYears} años para incidencias operativas, salvo obligación legal o reclamaciones en curso.` ,
    `Los registros de auditoría y seguridad se conservan por periodos limitados, orientativamente $ {logRetentionDays} días, salvo necesidad de investigación de incidentes o cumplimiento normativo.` ,
    "Al finalizar el servicio, el Encargado suprimirá o devolverá los datos personales, según instrucciones del Responsable, salvo obligación legal de conservación.",
    "Podrán mantenerse copias residuales en sistemas de respaldo por periodos limitados y bajo controles de seguridad.",
]);
```

```
hr();
drawH1("Anexo II · Medidas Técnicas y Organizativas (art. 32 RGPD)");
drawP(
    "Este anexo describe medidas orientativas aplicadas por el Encargado para proteger los datos tratados en la Plataforma. El nivel de medidas podrá variar según configuración, plan y alcance contratado, manteniendo un enfoque de seguridad razonable y proporcional al riesgo.",
);
drawH2("A. Control de acceso y autenticación");
drawList([
    "Principio de mínimos privilegios: cada usuario accede a lo necesario según su rol.",
    "Autenticación y gestión de sesiones con controles de expiración y revocación.",
    "Recomendación de contraseñas robustas y, cuando aplique, medidas adicionales (p. ej., MFA).",
]);
drawH2("B. Trazabilidad, auditoría y registros");
drawList([
    "Registro de eventos relevantes de seguridad y actividad (p. ej., accesos, operaciones sensibles, cambios de permisos).",
    "Protección razonable de logs y acceso restringido a personal autorizado.",
    "Retención limitada de logs conforme a necesidad operativa y seguridad.",
]);
drawH2("C. Cifrado y comunicaciones");
drawList(["Cifrado en tránsito mediante HTTPS/TLS.", "Uso de canales seguros para comunicaciones operativas y administrativas."]);
drawH2("D. Segregación y aislamiento");
drawList([
    "Separación lógica por organización cuando aplica (controles de acceso y políticas).",
    "Limitaciones de acceso a datos entre organizaciones y roles.",
]);
drawH2("E. Disponibilidad y resiliencia");
drawList([
    "Uso de infraestructura en la nube con capacidades de redundancia y continuidad (según proveedor).",
    "Copias de seguridad y mecanismos de recuperación ante incidencias (según configuración).",
]);
drawH2("F. Gestión de vulnerabilidades e incidentes");
drawList([
    "Procedimientos para identificar, contener y corregir incidentes de seguridad.",
```

```

    "Notificación al Responsable sin dilación indebida cuando una brecha afecte a datos personales, conforme
RGPD.",
  });
  drawH2("G. Confidencialidad y formación");
  drawList([
    "Compromisos de confidencialidad del personal con acceso a sistemas.",
    "Buenas prácticas y controles organizativos razonables para limitar accesos.",
  ]);
  drawH2("H. Minimización y buenas prácticas del Cliente");
  drawList([
    "El Cliente debe evitar registrar datos excesivos o no pertinentes.",
    "Evitar categorías especiales salvo estricta necesidad, base legal y garantías.",
    "Revisar periódicamente usuarios, roles y accesos.",
  ]);

  // ☒ Añadido: "no lista pública / no decisiones automatizadas"
  drawH2("I. Salvaguardas de uso profesional");
  drawList([
    "La Plataforma no es un registro público ni una lista de difusión; el Cliente no debe usarla como "lista
negra" pública ni para decisiones discriminatorias.",
    "Se recomienda que cualquier medida operativa relevante se base en revisión humana y evidencias internas del
Cliente (cuando existan).",
  ]);

  hr();
  drawH2("Contacto y ejercicio de derechos (cuando proceda)");
  drawP(
    `Para cuestiones de privacidad y seguridad: ${org.privacy_email}. Para cuestiones contractuales o del
servicio: ${org.provider_email}.`,
  );

  footer();
  return await pdfDoc.save();
}

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return preflight(req);
  if (req.method !== "POST") return json(req, 405, errPayload("method_not_allowed"));

  const started = Date.now();

  try {
    const body = (await req.json()).catch(() => null) as Body | null;
    const requestId = (body?.request_id ?? "").trim();
    const providedEmail = normalizeEmail(body?.email);

    if (!requestId) return badRequest(req, "missing_request_id");

    const supabase = supabaseServiceClient();

    // 1) Leer la solicitud (service role)
    // ⚠ IMPORTANTE: NO seleccionar columnas inexistentes (auth_user_id / request_token)
    const { data: row, error: readErr } = await supabase
      .from("debacu_eval_access_requests")

```

```

.select(
  `
    id,
    created_at,
    status,
    company_name,
    legal_name,
    cif,
    address,
    city,
    country,
    property_type,
    rooms_count,
    website,
    contact_name,
    contact_role,
    email,
    phone,
    notes,
    accepted_terms,
    accepted_terms_pdf_path,
    accepted_terms_pdf_sha256,
    accepted_terms_accepted_at,
    terms_version,
    accepted_terms_pdf_bucket
  `
)
.eq("id", requestId)
.maybeSingle();

```

```

if (readErr) {
  console.error("db_read_failed", {
    request_id: requestId,
    message: readErr.message,
    details: (readErr as any).details,
    hint: (readErr as any).hint,
    code: (readErr as any).code,
  });
  return serverError(req, "db_read_failed", readErr.message);
}

```

```

if (!row) return notFound(req);

```

```

const rowEmail = normalizeEmail((row as any).email);
const rowStatus = String((row as any).status ?? "").toUpperCase();

```

```

// 2) Autorización mínima SIN usuario:
// - Ideal: validar email del body contra email en BD
// - Si no viene email, limitar a status PENDING y created_at < 24h
if (providedEmail) {
  if (!rowEmail || providedEmail !== rowEmail) return forbidden(req, "email_mismatch");
} else {

```

```
// Sin email: no lo recomiendo, pero al menos no aceptes requests viejas o ya revisadas.
if (rowStatus !== "PENDING") return forbidden(req, "forbidden_status");
const createdAt = new Date((row as any).created_at);
const ageMs = Date.now() - createdAt.getTime();
if (!Number.isFinite(ageMs) || ageMs > 24 * 60 * 60 * 1000) {
    return forbidden(req, "request_expired");
}
}
```

```
// 3) Idempotencia
```

```
if ((row as any).accepted_terms === true && (row as any).accepted_terms_pdf_path) {
    return json(req, 200, {
        ok: true,
        proof: {
            request_id: (row as any).id,
            bucket: (row as any).accepted_terms_pdf_bucket ?? LEGAL_BUCKET,
            path: (row as any).accepted_terms_pdf_path,
            sha256: (row as any).accepted_terms_pdf_sha256,
            accepted_at: (row as any).accepted_terms_accepted_at,
            terms_version: (row as any).terms_version ?? TERMS_VERSION,
        },
    });
}
```

```
// 4) Validaciones mínimas para PDF coherente
```

```
if (!(row as any).company_name) return badRequest(req, "missing_company_name");
if (!(row as any).cif) return badRequest(req, "missing_cif");
if (!(row as any).contact_name) return badRequest(req, "missing_contact_name");
if (!(row as any).email) return badRequest(req, "missing_email");
```

```
// 5) Preparar PDF
```

```
const acceptedAt = nowIso();
const ip = getClientIp(req);
const userAgent = req.headers.get("user-agent") ?? null;
```

```
const pdfData: PdfData = {
    request_id: (row as any).id,
    terms_version: TERMS_VERSION,
```

```
    company_name: (row as any).company_name,
    legal_name: (row as any).legal_name,
    cif: (row as any).cif,
    address: (row as any).address,
    city: (row as any).city,
    country: (row as any).country,
    property_type: ((row as any).property_type as PropertyType | null) ?? null,
    rooms_count: (row as any).rooms_count,
    website: (row as any).website,
```

```
    contact_name: (row as any).contact_name,
    contact_role: (row as any).contact_role,
    email: (row as any).email,
```



```
phone: (row as any).phone,
```

```
notes: (row as any).notes,
```

```
accepted_ip: ip,
```

```
accepted_user_agent: userAgent,
```

```
// ☒ Políticas server-side que quedan reflejadas en el PDF (orientativas)
```

```
rgpd_retention_years: 3,
```

```
rgpd_log_retention_days: 180,
```

```
};
```

```
const pdfBytes = await buildPdf(pdfData, acceptedAt);
```

```
// 6) Hash SHA256 del PDF (integridad del fichero)
```

```
const digest = await crypto.subtle.digest("SHA-256", pdfBytes);
```

```
const sha256 = toHex(digest);
```

```
// 7) Upload a Storage (NO upsert)
```

```
const safeTs = safeTimestamp(acceptedAt);
```

```
const uniq = crypto.randomUUID();
```

```
const path = `debacu_eval/${(row as any).id}/terms_acceptance_${safeTs}_${uniq}.pdf`;
```

```
const { error: uploadError } = await supabase.storage
```

```
.from(LEGAL_BUCKET)
```

```
.upload(path, pdfBytes, { contentType: "application/pdf", upsert: false });
```

```
if (uploadError) {
```

```
  console.error("storage_upload_failed", {
```

```
    request_id: requestId,
```

```
    message: uploadError.message,
```

```
    name: (uploadError as any).name,
```

```
  });
```

```
  return serverError(req, "storage_upload_failed", uploadError.message);
```

```
}
```

```
// 8) Persistir aceptación en BD
```

```
// ☒ además marcamos DPA + Anexo II como aceptados (van incluidos dentro del mismo PDF)
```

```
// ☒ guardamos hash canónico del “paquete legal” (estable, no depende del PDF final)
```

```
const acceptanceLocale =
```

```
(req.headers.get("accept-language") || "").split(",")[0]?.trim() || "es-ES";
```

```
const canonicalDocString = [
```

```
  "DEBACU_EVAL_LEGAL_PACKAGE",
```

```
  `TERMS_VERSION=${TERMS_VERSION}`,
```

```
  `DPA_VERSION=${TERMS_VERSION}`,
```

```
  `RGPD_ANNEX_II_VERSION=${TERMS_VERSION}`,
```

```
  `RETENTION_YEARS=${String(pdfData.rgpd_retention_years ?? 3)}`,
```

```
  `LOG_RETENTION_DAYS=${String(pdfData.rgpd_log_retention_days ?? 180)}`,
```

```
  `LOCALE=${acceptanceLocale}`,
```

```
].join("|");
```

```

const canonicalDigest = await crypto.subtle.digest(
  "SHA-256",
  new TextEncoder().encode(canonicalDocString),
);
const canonicalHash = toHex(canonicalDigest);

const { error: updateError } = await supabase
  .from("debacu_eval_access_requests")
  .update({
    accepted_terms: true,
    accepted_terms_pdf_bucket: LEGAL_BUCKET,
    accepted_terms_pdf_path: path,
    accepted_terms_pdf_sha256: sha256,
    accepted_terms_accepted_at: acceptedAt,
    accepted_terms_at: acceptedAt, // ☒ lo estabas dejando null en tu tabla
    accepted_terms_ip: ip,
    accepted_terms_user_agent: userAgent,
    terms_version: TERMS_VERSION,

    // ☒ DPA (Encargo de tratamiento) – reutilizamos el mismo PDF
    dpa_accepted: true,
    dpa_version: TERMS_VERSION,
    dpa_accepted_at: acceptedAt,
    dpa_ip: ip,
    dpa_user_agent: userAgent,
    dpa_pdf_bucket: LEGAL_BUCKET,
    dpa_pdf_path: path,
    dpa_pdf_sha256: sha256,

    // ☒ RGPD Annex II – incluido en el mismo PDF
    rgpd_annex_ii_version: TERMS_VERSION,
    rgpd_annex_ii_accepted_at: acceptedAt,

    // ☒ Metadatos útiles para RGPD / evidencia
    acceptance_legal_basis: "ART_6_1_F_INTERES_LEGITIMO_PREVENCION_FRAUDE_Y_SEGURIDAD",
    acceptance_locale: acceptanceLocale,

    accepted_terms_doc_hash_algo: "SHA-256",
    accepted_terms_doc_hash: canonicalHash,
  })
  .eq("id", (row as any).id);

if (updateError) {
  console.error("db_update_failed", {
    request_id: requestId,
    message: updateError.message,
    details: (updateError as any).details,
    hint: (updateError as any).hint,
    code: (updateError as any).code,
  });

  // best-effort cleanup para evitar orphan files

```

```

    try {
      await supabase.storage.from(LEGAL_BUCKET).remove([path]);
    } catch {
      // ignore
    }

    return serverError(req, "db_update_failed", updateError.message);
  }

  console.log("ok", { request_id: requestId, ms: Date.now() - started });

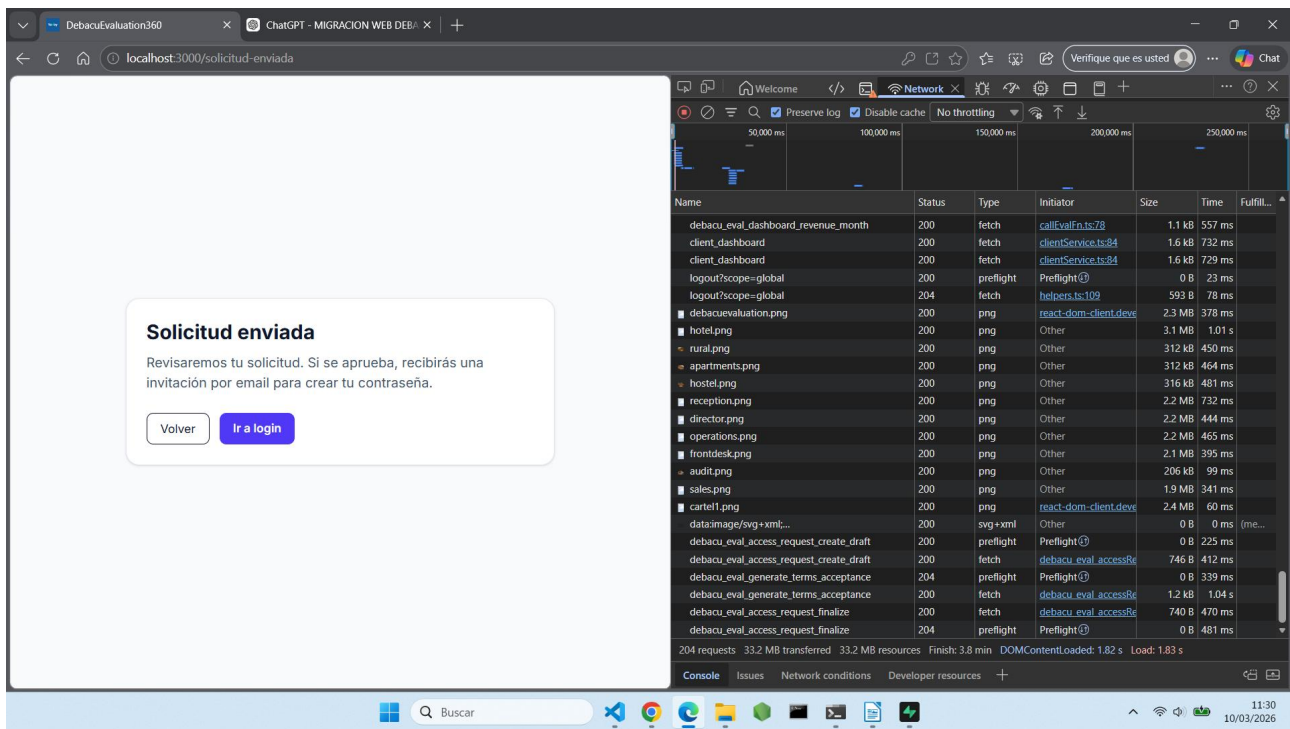
  return json(req, 200, {
    ok: true,
    proof: {
      request_id: (row as any).id,
      bucket: LEGAL_BUCKET,
      path,
      sha256,
      accepted_at: acceptedAt,
      terms_version: TERMS_VERSION,

      // ☒ extra evidencia útil (no rompe compatibilidad)
      dpa_accepted: true,
      dpa_version: TERMS_VERSION,
      ngpd_annex_ii_version: TERMS_VERSION,
      accepted_terms_doc_hash: canonicalHash,
      accepted_terms_doc_hash_algo: "SHA-256",
      acceptance_locale: acceptanceLocale,
    },
  });
} catch (e: any) {
  console.error("internal_error", {
    message: e?.message ?? String(e),
    stack: e?.stack,
    ms: Date.now() - started,
  });
  return serverError(req, "internal_error", e?.message ?? String(e));
}
});

```

=====

FINALIZA Y GRABA EN BD



```
FUNCION // supabase/functions/debacu_eval_access_request_set_professional_use/index.ts
```

```
import "jsr:@supabase/functions-js/edge-runtime.d.ts";
import { json, preflight } from "../_shared/cors.ts";
import { supabaseServiceClient } from "../_shared/auth.ts";
```

```
type Body = {
  request_id: string;
  accepted_professional_use: boolean;
};
```

```
async function readJsonSafe<T>(req: Request): Promise<T | null> {
  try {
    const text = await req.text();
    if (!text) return null;
    return JSON.parse(text) as T;
  } catch {
    return null;
  }
}
```

```
Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return preflight(req);
  if (req.method !== "POST") {
    return json(req, 405, {
      ok: false,
      error: "request_failed",
      detail: "METHOD_NOT_ALLOWED",
    });
  }
}
```

```
const body = await readJsonSafe<Body>(req);
if (!body) {
```

```

return json(req, 400, {
  ok: false,
  error: "request_failed",
  detail: "invalid_json",
});
}

```

```

const request_id = String(body.request_id ?? "").trim();
if (!request_id) {
  return json(req, 400, {
    ok: false,
    error: "request_failed",
    detail: "missing_request_id",
  });
}

```

```

const supabase = supabaseServiceClient();

```

```

try {
  // 1) comprobar que existe y que hay prueba PDF
  const { data: row, error: selErr } = await supabase
    .from("debacu_eval_access_requests")
    .select("id, status, accepted_terms, accepted_terms_pdf_path")
    .eq("id", request_id)
    .maybeSingle();

```

```

  if (selErr) {
    return json(req, 500, {
      ok: false,
      error: "request_failed",
      detail: "DB_READ_FAILED",
    });
  }
  if (!row) {
    return json(req, 404, {
      ok: false,
      error: "request_failed",
      detail: "NOT_FOUND",
    });
  }
}

```

```

  if (!row.accepted_terms || !row.accepted_terms_pdf_path) {
    return json(req, 400, {
      ok: false,
      error: "request_failed",
      detail: "terms_not_accepted_with_proof",
    });
  }
}

```

```

// 2) update (sin tocar stack traces / sin revelar mensajes)
const { error: updErr } = await supabase
  .from("debacu_eval_access_requests")

```

```

.update({
  // si tu flujo exige "volver a pending" lo mantengo,
  // si no, bórralo para evitar pisar estados.
  status: "PENDING",
  accepted_professional_use: !!body.accepted_professional_use,
})
.eq("id", request_id);

if (updErr) {
  return json(req, 500, {
    ok: false,
    error: "request_failed",
    detail: "DB_UPDATE_FAILED",
  });
}

return json(req, 200, { ok: true });
} catch {
  return json(req, 500, {
    ok: false,
    error: "request_failed",
    detail: "INTERNAL_ERROR",
  });
}
});

```

AQUI YA SE HA TERMINADO LA SOLICITUD DE ACCESO, POSTERIORMENTE EL ADMIN ENTRA EN SU DASHBOARD Y ACCEDE A LA INFORMACION

ADMIN ENTRA

The screenshot displays a web application interface for an administrator. The left sidebar contains navigation links: Dashboard, Solicitudes (active), Clientes, Planes, Facturación, Uso y abuso, Estadísticas, Auditoría, Exportaciones, Cambios, and Configuración. The main content area is titled 'Panel de administración' and 'Solicitudes de acceso'. It shows a list of access requests with columns for Fecha, Empresa, CIF, ResponsableEmail, and Acción. One request is highlighted for 'HOTEL2' on 10/3/2026, with a status of 'Pendiente'. The action buttons are 'Aprobar' (green) and 'Rechazar' (red). On the right, a network inspector shows a list of requests, including 'cartel1.png', 'data:image/svg+xml...', and various API calls related to 'debacu_eval_access_request'.

SE CARGAN LAS SOLICITUDES PENDIENTES

```
// supabase/functions/debacu-eval-admin-access-requests/index.ts
// deno-lint-ignore-file no-explicit-any
import "jsr:@supabase/functions-js/edge-runtime.d.ts";

import { createClient } from "https://esm.sh/@supabase/supabase-js@2.45.4";
import { json, preflight } from "../_shared/cors.ts";
import { requireAdmin } from "../_shared/auth.ts";

/* =====
 * Env
 * ===== */
function mustEnv(name: string) {
  const v = Deno.env.get(name);
  if (!v) throw new Error(`MISSING_ENV:${name}`);
  return v;
}

const SUPABASE_URL = mustEnv("SUPABASE_URL");
const SERVICE_ROLE_KEY = mustEnv("SUPABASE_SERVICE_ROLE_KEY");

/**
 * URL base de tu front (producción).
 * Ej: https://www.debacu.com
 * Si no está, intentamos inferir del Origin.
 */
const APP_PUBLIC_URL = Deno.env.get("DEBACU_APP_URL") || Deno.env.get("SITE_URL") || "";

// Ajusta si tu tabla se llama diferente
const ACCESS_REQUESTS_TABLE = "debacu_eval_access_requests";
const APP_ID = "DEBACU_EVAL";

/* =====
 * Helpers
 * ===== */
function safeUpper(v: any) {
  return String(v ?? "").toUpperCase().trim();
}
function safeStr(v: any) {
  return typeof v === "string" ? v.trim() : "";
}
function isUuid(v: string) {
  return /^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i.test(v);
}
function pickString(body: any, snake: string, camel?: string) {
  const v = body?.[snake] ?? (camel ? body?.[camel] : undefined);
  return typeof v === "string" && v.trim() ? v.trim() : undefined;
}
function pickBool(body: any, snake: string, camel?: string) {
  const v = body?.[snake] ?? (camel ? body?.[camel] : undefined);
  return Boolean(v);
}
function pickLimit(body: any, def = 100) {
  const n = Number(body?.limit ?? def);
  if (!Number.isFinite(n) || n <= 0) return def;
  return Math.min(Math.floor(n), 500);
}
```

```

function nowIso() {
  return new Date().toISOString();
}
function toISODate(d: Date) {
  return d.toISOString().slice(0, 10);
}
function addDaysISO(isoDate: string, days: number) {
  const [y, m, dd] = isoDate.split("-").map((x) => Number(x));
  const dt = new Date(Date.UTC(y, (m - 1), dd, 0, 0, 0));
  dt.setUTCDate(dt.getUTCDate() + days);
  return toISODate(dt);
}
function normEmail(email: string) {
  return safeStr(email).toLowerCase();
}

function sbService() {
  return createClient(SUPABASE_URL, SERVICE_ROLE_KEY, {
    auth: { persistSession: false, autoRefreshToken: false },
  });
}

function err(req: Request, status: number, detail: string, extra?: Record<string, unknown>) {
  return json(req, status, { ok: false, error: "request_failed", detail, ...(extra ?? {}) });
}

function inferAppUrl(req: Request) {
  if (APP_PUBLIC_URL && APP_PUBLIC_URL.startsWith("http")) return APP_PUBLIC_URL;

  const origin = req.headers.get("origin") || "";
  if (origin.startsWith("http")) return origin;

  const xfHost = req.headers.get("x-forwarded-host") || req.headers.get("host") || "";
  const xfProto = req.headers.get("x-forwarded-proto") || "https";
  if (xfHost) return `${xfProto}://${xfHost}`;

  return ""; // último recurso
}

function buildInviteRedirect(req: Request, orgId: string, requestId: string) {
  const base = inferAppUrl(req);
  const path = `/auth/activate?org_id=${encodeURIComponent(orgId)}&req=${encodeURIComponent(requestId)}`;
  return base ? `${base}${path}` : path;
}

/* =====
 * Extract fields from request row (tolerante a esquemas)
 * ===== */
function getReqField(row: any, ...keys: string[]) {
  for (const k of keys) {
    const v = row?.[k];
    if (typeof v === "string" && v.trim()) return v.trim();
    if (typeof v === "number" && Number.isFinite(v)) return v;
  }
  return null;
}

```



```

function mapCustomerInsert(row: any) {
  const email = normEmail(String(row.email || ""));
  const name =
    getReqField(row, "customer_name", "hotel_name", "company_name", "name", "commercial_name") ?? email;

  const nif = getReqField(row, "nif", "cif") ?? null;
  const address = getReqField(row, "address") ?? null;
  const city = getReqField(row, "city") ?? null;
  const country = getReqField(row, "country") ?? "ESP";
  const phone = getReqField(row, "phone") ?? null;

  return {
    name,
    nif,
    address,
    city,
    country,
    phone,
    email,
    app_id: APP_ID,
    is_active: true,
    updated_at: nowIso(),
  } as any;
}

```

```

function mapOrgInsert(row: any, customer_id: string) {
  const name =
    getReqField(row, "org_name", "hotel_name", "company_name", "name") ??
    `Hotel ${normEmail(String(row.email || ""))}`;

  const legal_name = getReqField(row, "legal_name") ?? null;
  const cif = getReqField(row, "cif", "nif") ?? null;
  const address = getReqField(row, "address") ?? null;
  const city = getReqField(row, "city") ?? null;
  const country = getReqField(row, "country") ?? "ESP";
  const website = getReqField(row, "website") ?? null;

  const property_type = safeUpper(getReqField(row, "property_type") ?? "HOTEL");
  const rooms_count = Number(getReqField(row, "rooms_count") ?? 0) || null;

  return {
    name,
    legal_name,
    cif,
    address,
    city,
    country,
    website,
    property_type,
    rooms_count,
    customer_id,
    created_at: nowIso(),
  }
}

```

```

    } as any;
}

/* =====
* DB primitives
* ===== */
async function getAccessRequest(sb: ReturnType<typeof sbService>, requestId: string) {
  const { data, error } = await sb
    .from(ACCESS_REQUESTS_TABLE)
    .select("*")
    .eq("id", requestId)
    .maybeSingle();
  if (error) throw error;
  if (!data) throw new Error("not_found");
  return data as any;
}

async function findCustomerByEmail(sb: ReturnType<typeof sbService>, email: string) {
  const { data, error } = await sb
    .from("customers")
    .select("*")
    .eq("app_id", APP_ID)
    .ilike("email", email)
    .order("created_at", { ascending: false })
    .limit(1)
    .maybeSingle();

  if (error) throw error;
  return data as any | null;
}

async function ensureCustomer(sb: ReturnType<typeof sbService>, row: any, auth_user_id?: string | null) {
  const existingId = row.customer_id ? String(row.customer_id) : null;
  if (existingId && isUuid(existingId)) {
    const { data, error } = await sb.from("customers").select("*").eq("id", existingId).maybeSingle();
    if (error) throw error;
    if (data) return data as any;
  }

  const email = normEmail(String(row.email || ""));
  const found = await findCustomerByEmail(sb, email);
  if (found) {
    if (auth_user_id && !found.auth_user_id) {
      const { error } = await sb
        .from("customers")
        .update({ auth_user_id, updated_at: nowIso() } as any)
        .eq("id", found.id);
      if (error) throw error;
      return { ...found, auth_user_id };
    }
    return found;
  }

  const payload = mapCustomerInsert(row);

```

```

    if (auth_user_id) payload.auth_user_id = auth_user_id;

    const { data: created, error: insErr } = await sb.from("customers").insert(payload).select("*").maybeSingle();
    if (insErr) throw insErr;
    if (!created) throw new Error("customer_create_failed");
    return created as any;
}

async function ensureOrg(sb: ReturnType<typeof sbService>, row: any, customer_id: string) {
    const existingId = row.org_id ? String(row.org_id) : null;
    if (existingId && isUuid(existingId)) {
        const { data, error } = await sb.from("debacu_eval_organizations").select("*").eq("id",
existingId).maybeSingle();
        if (error) throw error;
        if (data) return data as any;
    }

    const { data: found, error: fErr } = await sb
        .from("debacu_eval_organizations")
        .select("*")
        .eq("customer_id", customer_id)
        .order("created_at", { ascending: false })
        .limit(1)
        .maybeSingle();

    if (fErr) throw fErr;
    if (found) return found as any;

    const payload = mapOrgInsert(row, customer_id);
    const { data: created, error: insErr } = await sb
        .from("debacu_eval_organizations")
        .insert(payload)
        .select("*")
        .maybeSingle();

    if (insErr) throw insErr;
    if (!created) throw new Error("org_create_failed");
    return created as any;
}

async function ensureOrgMemberOwnerInvited(
    sb: ReturnType<typeof sbService>,
    params: {
        org_id: string;
        invited_email: string;
        admin_user_id: string;
        auth_user_id?: string | null;
    },
) {
    const { org_id, invited_email, admin_user_id, auth_user_id } = params;
    const email = normEmail(invited_email);

    let q = sb
        .from("debacu_eval_org_members")

```

```

        .select("*")
        .eq("org_id", org_id)
        .order("created_at", { ascending: false })
        .limit(1);

    if (auth_user_id) q = q.or(`auth_user_id.eq.${auth_user_id},user_id.eq.${auth_user_id},invited_email.ilike.${email}`);
    else q = q.ilike("invited_email", email);

    const { data: existing, error: exErr } = await q.maybeSingle();
    if (exErr) throw exErr;

    const patch: any = {
        role: "OWNER",
        status: "INVITED",
        invited_email: email,
        created_by_user_id: admin_user_id,
        updated_at: nowIso(),
    };

    if (auth_user_id) {
        patch.user_id = auth_user_id;
        patch.auth_user_id = auth_user_id;
    }

    if (existing?.id) {
        const { data: upd, error } = await sb
            .from("debacu_eval_org_members")
            .update(patch)
            .eq("id", existing.id)
            .select("*")
            .maybeSingle();
        if (error) throw error;
        return upd ?? existing;
    }

    const insertPayload: any = { org_id, ...patch, created_at: nowIso() };

    const { data: created, error: insErr } = await sb
        .from("debacu_eval_org_members")
        .insert(insertPayload)
        .select("*")
        .maybeSingle();

    if (insErr) throw insErr;
    if (!created) throw new Error("org_member_create_failed");
    return created as any;
}

/**
 * OJO: tu tabla debacu_eval_access_requests NO tiene updated_at (según tu dump).
 */

```

```

async function updateAccessRequest(sb: ReturnType<typeof sbService>, requestId: string, patch: Record<string, any>) {
  const { data, error } = await sb
    .from(ACCESS_REQUESTS_TABLE)
    .update({ ...patch } as any)
    .eq("id", requestId)
    .select("*")
    .maybeSingle();

  if (error) throw error;
  return data as any;
}

/* =====
 * Subscriptions: asegurar FREE_TRIAL al aprobar
 * ===== */
async function getPlanByCode(sb: ReturnType<typeof sbService>, planCode: string) {
  const { data, error } = await sb
    .from("plans")
    .select("id, code, app_id")
    .eq("app_id", APP_ID)
    .eq("code", planCode)
    .maybeSingle();
  if (error) throw error;
  if (!data?.id) throw new Error(`plan_not_found:${planCode}`);
  return data as { id: string; code: string; app_id: string };
}

async function findActiveLikeSubscription(sb: ReturnType<typeof sbService>, customerId: string) {
  const today = toISODate(new Date());

  const { data, error } = await sb
    .from("subscriptions")
    .select("*")
    .eq("app_id", APP_ID)
    .eq("customer_id", customerId)
    .in("status", ["ACTIVE", "TRIAL_ACTIVE"])
    .order("created_at", { ascending: false })
    .limit(10);

  if (error) throw error;
  const rows = (data ?? []) as any[];

  const valid = rows.find((s) => {
    const end = typeof s?.end_date === "string" ? s.end_date : null;
    return !end || end >= today;
  });

  return valid ?? null;
}

async function ensureFreeTrialSubscription(sb: ReturnType<typeof sbService>, customerId: string) {
  const existing = await findActiveLikeSubscription(sb, customerId);
  if (existing) return { created: false, subscription: existing };
}

```

```

const plan = await getPlanByCode(sb, "FREE");

const start = toISODate(new Date());
const end = addDaysISO(start, 30);

const payload: any = {
  customer_id: customerId,
  app_id: APP_ID,
  plan_id: plan.id,
  billing_frequency: "FREE_TRIAL",
  start_date: start,
  end_date: end,
  next_billing_date: end,
  status: "TRIAL_ACTIVE",
  provider: "manual",
  extra_seats: 0,
  created_at: nowIso(),
  updated_at: nowIso(),
};

const { data: created, error: insErr } = await sb
  .from("subscriptions")
  .insert(payload)
  .select("*")
  .maybeSingle();

if (insErr) throw insErr;
if (!created) throw new Error("subscription_create_failed");

return { created: true, subscription: created };
}

/* =====
* Import Profiles: asegurar perfil CSV por org al aprobar
* ===== */
const DEFAULT_IMPORT_PROFILE_NAME = "CSV_STANDARD_TEST";
const DEFAULT_IMPORT_SOURCE_TYPE = "FUTURE_BOOKINGS";

// mapping estándar que ya usas en el CSV de test
const DEFAULT_IMPORT_MAPPING = {
  email: "email",
  phone: "phone",
  country: "country",
  document: "document_number",
  full_name: "full_name",
  checkin_date: "checkin_date",
  checkout_date: "checkout_date",
};

async function ensureDefaultImportProfileForOrg(sb: ReturnType<typeof sbService>, orgId: string) {
  // si ya existe un perfil para ese org (por nombre), no creamos otro
  const { data: found, error: fErr } = await sb
    .from("import_profiles")

```

```

        .select("*")
        .eq("org_id", orgId)
        .eq("name", DEFAULT_IMPORT_PROFILE_NAME)
        .order("created_at", { ascending: false })
        .limit(1)
        .maybeSingle();

    if (fErr) throw fErr;
    if (found?.id) return { created: false, import_profile: found };

    const payload: any = {
        org_id: orgId,
        name: DEFAULT_IMPORT_PROFILE_NAME,
        source_type: DEFAULT_IMPORT_SOURCE_TYPE, // FUTURE_BOOKINGS
        delimiter: ",",
        date_format: "YYYY-MM-DD",
        decimal_separator: ".",
        encoding: "UTF-8",
        identity_strategy: "DOCUMENT_STRONG",
        mapping: DEFAULT_IMPORT_MAPPING,
        disabled_fields: [],
        created_at: nowIso(),
        updated_at: nowIso(),
    };

    const { data: created, error: insErr } = await sb
        .from("import_profiles")
        .insert(payload)
        .select("*")
        .maybeSingle();

    if (insErr) throw insErr;
    if (!created) throw new Error("import_profile_create_failed");

    return { created: true, import_profile: created };
}

/* =====
 * Email send helper (INVITE o RECOVERY)
 * ===== */
async function sendAuthEmailWithRedirect(
    sb: ReturnType<typeof sbService>,
    email: string,
    redirectTo: string,
    extraData?: Record<string, any>,
) {
    const { data: inviteData, error: inviteErr } = await sb.auth.admin.inviteUserByEmail(email, {
        redirectTo,
        data: extraData ?? {},
    });

    if (!inviteErr) {
        return {

```

```

    ok: true,
    action: "INVITE" as const,
    auth_user_id: inviteData?.user?.id ?? null,
    detail: inviteData?.user?.id ? `INVITE (${inviteData.user.id})` : "INVITE (ok)",
  };
}

const msg = String(inviteErr?.message ?? "").toLowerCase();
const looksLikeExists = msg.includes("already") || msg.includes("registered") || msg.includes("exists");

if (!looksLikeExists) {
  return { ok: false, action: "ERROR" as const, auth_user_id: null, detail: String(inviteErr.message ?? inviteErr) };
}

const { error: recErr } = await sb.auth.admin.resetPasswordForEmail(email, { redirectTo });
if (recErr) {
  return { ok: false, action: "ERROR" as const, auth_user_id: null, detail: String(recErr.message ?? recErr) };
}

return { ok: true, action: "RECOVERY" as const, auth_user_id: null, detail: "RECOVERY (ok)" };
}

/* =====
* Main
* ===== */
Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return preflight(req);
  if (req.method !== "POST") return json(req, 405, { ok: false, error: "method_not_allowed" });

  const sb = sbService();

  try {
    const admin = await requireAdmin(req);
    if (!admin?.user_id) return err(req, 401, "UNAUTHENTICATED");

    const body = await req.json().catch(() => ({}));
    const action = safeUpper(body?.action);
    if (!action) return err(req, 400, "missing_action");

    /* =====
    * LIST
    * ===== */
    if (action === "LIST") {
      const status = safeUpper(body?.status ?? "PENDING"); // PENDING | APPROVED | REJECTED | ALL
      const limit = pickLimit(body, 100);

      let q = sb
        .from(ACCESS_REQUESTS_TABLE)
        .select("*")
        .order("created_at", { ascending: false })
        .limit(limit);

```



```

    if (status && status !== "ALL") q = q.eq("status", status);

    const { data, error } = await q;
    if (error) throw error;

    return json(req, 200, { ok: true, data: data ?? [] });
}

/* =====
 * APPROVE / REJECT / RESEND
 * ===== */
const requestId = pickString(body, "requestId", "request_id");
if (!requestId) return err(req, 400, "missing_requestId");

const row = await getAccessRequest(sb, requestId);
const email = normEmail(String(row.email || ""));
if (!email) return err(req, 400, "request_missing_email");

const decisionNotes = pickString(body, "decisionNotes", "decision_notes") ?? null;

// REJECT
if (action === "REJECT") {
    const updated = await updateAccessRequest(sb, requestId, {
        status: "REJECTED",
        decision_notes: decisionNotes,
        reviewed_by: admin.user_id,
        reviewed_at: nowIso(),
        last_email_status: "SKIPPED",
        last_email_at: nowIso(),
        last_email_detail: "Reject: no email sent",
    });

    return json(req, 200, { ok: true, request: updated });
}

if (action !== "APPROVE" && action !== "RESEND") return err(req, 400, "invalid_action");

// 1) Customer
const customer = await ensureCustomer(sb, row, null);

// 1bis) Subscription FREE_TRIAL (evita PLAN_NOT_ACTIVE)
const subRes = await ensureFreeTrialSubscription(sb, String(customer.id));

// 2) Org
const org = await ensureOrg(sb, row, String(customer.id));

// ☑ 2bis) Import profile por defecto del org (evita selector vacío en CSV)
const importRes = await ensureDefaultImportProfileForOrg(sb, String(org.id));

// 3) Redirect
const invite_redirect_to = buildInviteRedirect(req, String(org.id), requestId);

```

```

// 3bis) Data para template (si lo usas)
const invite_data = { org_id: String(org.id), req: String(requestId) };

// 4) Email (INVITE o RECOVERY)
const sendRes = await sendAuthEmailWithRedirect(sb, email, invite_redirect_to, invite_data);

// 5) Org member (OWNER INVITED)
let member: any = null;
if (sendRes.ok && sendRes.auth_user_id) {
  if (!customer.auth_user_id) {
    await sb
      .from("customers")
      .update({ auth_user_id: sendRes.auth_user_id, updated_at: nowIso() } as any)
      .eq("id", customer.id);
  }
  member = await ensureOrgMemberOwnerInvited(sb, {
    org_id: String(org.id),
    invited_email: email,
    admin_user_id: admin.user_id,
    auth_user_id: sendRes.auth_user_id,
  });
} else {
  member = await ensureOrgMemberOwnerInvited(sb, {
    org_id: String(org.id),
    invited_email: email,
    admin_user_id: admin.user_id,
    auth_user_id: null,
  });
}

// 6) Cerrar otras pendientes (opcional)
const closeOtherPendings = pickBool(body, "closeOtherPendings", "close_other_pendings");
if (closeOtherPendings) {
  await sb
    .from(AccessRequestsTable)
    .update({
      status: "SUPERSEDED",
      decision_notes: "Auto-superseded by another approval.",
      reviewed_by: admin.user_id,
      reviewed_at: nowIso(),
    } as any)
    .neq("id", requestId)
    .ilike("email", email)
    .eq("status", "PENDING");
}

// 7) Update access_request
const newStatus =
  action === "APPROVE" ? "APPROVED" : safeUpper(row.status) === "APPROVED" ? "APPROVED" : "PENDING";

const updated = await updateAccessRequest(sb, requestId, {
  status: newStatus,

```

```

customer_id: customer.id,
org_id: org.id,
decision_notes: action === "APPROVE" ? decisionNotes : row.decision_notes ?? null,
reviewed_by: action === "APPROVE" ? admin.user_id : row.reviewed_by ?? null,
reviewed_at: action === "APPROVE" ? nowIso() : row.reviewed_at ?? null,
last_email_status: sendRes.ok ? "SENT" : "FAILED",
last_email_at: nowIso(),
last_email_detail: sendRes.ok
  ? `${sendRes.action} | ${sendRes.detail} | redirect=${invite_redirect_to}`
  : sendRes.detail,
});

```

```

// 8) Response

```

```

return json(req, 200, {
  ok: true,
  action,
  created_or_reused: {
    auth_user_id: sendRes.auth_user_id ?? null,
    customer_id: customer.id,
    org_id: org.id,
    org_member_id: member?.id ?? null,

    subscription_id: subRes?.subscription?.id ?? null,
    subscription_created: Boolean(subRes?.created),
    subscription_status: subRes?.subscription?.status ?? null,
    subscription_end_date: subRes?.subscription?.end_date ?? null,

```

```

    import_profile_id: importRes?.import_profile?.id ?? null,
    import_profile_created: Boolean(importRes?.created),
    import_profile_name: importRes?.import_profile?.name ?? null,
  },
  invite: {
    email,
    redirect_to: invite_redirect_to,
    sent: sendRes.ok,
    email_action: sendRes.ok ? sendRes.action : "ERROR",
    detail: sendRes.detail,
  },
  request: updated,
});
} catch (e: any) {
  console.error("debacu-eval-admin-access-requests error:", e);
  const msg = String(e?.message ?? e);

```

```

  if (msg.includes("MISSING_ENV:")) {
    return json(req, 500, { ok: false, error: "request_failed", detail: msg });
  }
  if (msg === "not_found") return err(req, 404, "not_found");

```

```

  return json(req, 500, { ok: false, error: "request_failed", detail: msg });
}
});

```

se cargan las pendientes

y al aceptar la solicitud se crea el usuario en bd, la organización, members, perfiles, tablas para el revenue,etc...

tendremos que distinguir para no tocar todo, que con el nuevo formulario vienen muchos mas datos, para dar acceso solo necesitamos los actuales, el resto son datos que posteriormente usaremos en revenue y tendremos que ver como los canalizamos a cada sitio pero de momento con modificar la tabla actual donde se reciben los accesos nos vale después ya veremos que hacemos con el resto de datos.